

1. Représentation des données

HTTP: entêtes + corps

↑
Flot d'octets

de flot d'octets est une **erreur historique**
 → En réalité ce que l'on veut c'est un
 flot de messages. alors que TCP implémente
 un flot d'octets donc HTTP (qui utilise
 TCP). (Parenthèse SCTP transfère
 un flot de message)

→ Codage

1.1 codage déjà fait

Ex: JPEG, PNG

→ bien défini
 → stable
 → extensible
 → réalisation

format avec
 perte de données
 pas visible à l'œil
 nu

↓
 qui fait tout mais
 moins efficace

mais très efficace

de transfert de JPEG ou PNG

ne sont pas défini il faut passer par
JFIF

Ex2: Transfert de texte $\rightarrow$ UTF-8
Composition.
présentation
fin de lignes.

↑
inventer
pour
aller par
les m. person
c'moi n'inven
ter

1.2 / Codage ad hoc (pour cela)
Ex: identificateurs: liste de chaînes
 $\rightarrow$ terminées par '\n'
1.2.1) Textuel LL(1) descente

recursive

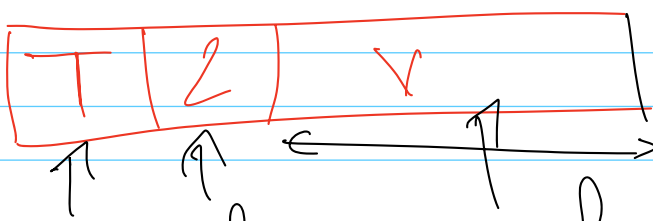
mais inefficace

1.2.2) TLV

Info to to . codage longueur valeur

.....
↑ un terminateur

examinelle erreur



Type longueur valeur

ProtoBuf: utiliser par exemple par Google

→ Def d'interface ~~protoc~~
 ↓
 (Def de struct)
 compilateur

ProtoBuf difficile de maintenir la compatibilité entre version

1.3 / Formats textuels Génériques

But: Format universel

→ facile à lire par machine

→ facile pour l'humain.

13.1 XML

SGML → HTML application SGML
 ↓
 texte enrichie

XML → structures de données

1.3.2 : JSON

nb 1 [1, 1] 7ab

string : "lolo", "Dis" {"a": 42}.

Pb1 : La sémantique des Nombres :
nombre Javascript

Coder un entier ≤ 53 bits

Pb2 : non-typé :

[1, "lolo", true, {"a": 1}]
"chaîne JS UTF-16"

↓ windows Java JS-

(Parenthèse U(-1)) on connaît
le type des données avant
de parcourir toutes les données
Pour transférer 4 octets
JSON on utilise

[123, 12, 18, 125] 16 caractères
base dix "78 A05 503" $\times 4 \times 2$

base 16

"base 64"

$\times \frac{4}{3}$

- inefficience données binaires
- pas parfaitement streamable

1.3. 2.1 380M lines: JSON

- Lignes, chaque ligne → 380M
- streamable (ligne par ligne)
- append

1.4 Format génériques binaires

CBOR → le mieux fait
(TLV)

2/ Structure des protocoles

2.1) XML-RPC, SOAP

main

f()

f

Remote proc
↑
RPC

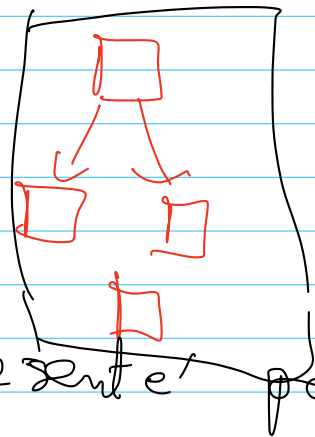
remplacer la
prosecur

le protocole RMI : ... - In

2.2 REST : base sur le transfert des ~~vals~~ des objets representation

C

S



- Chaque val sera représenté par une URL distincte

- Les relations par des hyperliens

Accede aux ressources : méthode

HTTP (standard)

lire une ressource → GET

modifier ou créer une ressource → PUT

supprimer une ressource → DELETE

Créer une ressource et
laisser le serveur la nommer } → POST

... /chat/ . GET → list d'hyperliens

/chat/id : Get → message

créer

PUT /chat/id ← côté client

POST /chat/ nommage

côté serveur

limitation: limite au transfert
de valeurs

le nbre de requêtes est exponentiel